

QA 노트

목차

1. TypeScript 타입 정의 정리
2. 버그: home.tsx prop 이름 대소문자 오타
3. 로그인 상태 관리: useState만으로 충분할까?
4. Supabase Authentication vs profiles 테이블
5. profiles 테이블 SQL 설계
6. signUp 폼 연결 순서 (8단계 로드맵)
7. handleSubmit 함수 뜯어보기
8. 실제 회원가입 요청 보내기 (supabase.auth.signUp)
9. 로딩 상태 관리 (isLoading)
10. 화면에 로딩 표시하기
11. redirect() vs useRouter()
12. 로그인 페이지 연결 순서
13. 에러 메시지 상태 (errorMessage)
14. AuthContext로 로그인 상태를 앱 전체에 공유하기
15. 트러블슈팅: useAuth is not a function
16. Header에 로그인/로그아웃 버튼 조건부 표시하기

1. TypeScript 타입 정의 정리

src/types/property.ts

```
export type Room = {  
  label: string;  
  name: string;  
  price: string;  
  description: string;  
  img: string;  
};  
  
export type RoomWithMeta = Room & {  
  like: boolean;  
  score: number;  
};  
  
export type RoomList = RoomWithMeta[];  
  
export type PropertyProps = {  
  roomType: Room[];  
};
```

`{}` 를 쓰는 곳 vs 안 쓰는 곳

타입	<code>{}</code> 사용 여부	이유
<code>Room</code>	O	새 객체 모양(필드 목록)을 처음부터 정의
<code>RoomWithMeta</code>	O (& 뒤에)	<code>Room</code> 에 필드(<code>like</code> , <code>score</code>)를 "추가"하는데, 추가할 필드 목록을 적어야 하니까
<code>RoomList</code>	X	<code>RoomWithMeta[]</code> 는 이미 있는 타입을 배열로 감싸기만 함. 새 필드가 없으니 <code>{}</code> 불필요
<code>PropertyProps</code>	O	새 객체 모양(필드 목록)을 처음부터 정의

정확한 규칙: `{}`는 "&가 있어서"가 아니라 "그 자리에서 새 객체 모양을 즉석으로 적을 때" 필요하다.

```
type Meta = { like: boolean; score: number };
type RoomWithMeta = Room & Meta; // 둘 다 이미 있는 타입 → {} 없음
type RoomWithMeta = Room & { like: boolean }; // 즉석으로 필드 적음 → {} 필요
```

이미 이름 붙은 타입끼리 &로 합칠 땐 `{}`가 필요 없다. &와 `{}`가 자주 같이 나오는 이유는, 새 필드 몇 개를 위해 따로 이름 붙이기 귀찮아서 바로 인라인으로 적기 때문일 뿐.

비유 (중학생용)

- `type`은 "이런 모양의 물건을 만들 거야"라는 설계도
- `{}`는 "여기 안에 필요한 걸 다 적을게"라는 상자
- &는 "그리고"라는 뜻 → 기존 설계도 + 새로 적은 것을 합침
- `[]`는 "이 모양의 물건이 여러 개 모인 배열"

한 줄 정리: 새 설계를 그릴 때만 상자(`{}`)를 쓰고, 이미 그려둔 설계를 그냥 가져다 쓰거나 배열로 묶기만 할 때는 상자가 필요 없다.

2. 버그: home.tsx prop 이름 대소문자 오타

`src/components/main/home.tsx`에서:

```
<KakaoMap RoomList={fakeDate} /> // ❌ 대문자 R
```

`KakaoMap` 컴포넌트는 `roomList`(소문자 시작)를 받도록 정의되어 있어서, 대소문자가 다르면 다른 이름의 prop으로 취급되어 타입 에러 발생. `roomList={fakeDate}`로 수정 완료.

3. 로그인 상태 관리: useState만으로 충분할까?

로컬 `useState(true/false)`만으로는 부족하다.

이유

- 새로고침하면 `useState` 값은 초기화되지만, Supabase 세션(쿠키)은 그대로 살아있음 → 화면 상태와 실제 로그인 상태가 어긋남
- `src/services/supabase/middleware.ts`에서 이미 `supabase.auth.getUser()`로 세션을 서버에서 관리 중. "로그인 됐냐"의 진짜 정답은 Supabase 세션이 갖고 있고, `useState`는 그걸 복제만 하는 셈

권장 방식

1. 전역 로그인 여부가 필요하면 `AuthContext`를 만들어서 `supabase.auth.onAuthStateChange()`로 세션 변화를 구독
2. 또는 Server Component에서 `server.ts`의 `createClient()`로 매번 `auth.getUser()` 확인해서 서버에서 분기 (더 안전, 새로고침해도 항상 정확)

단순 프로토타입이면 `useState`로 임시로 해도 되지만, 진짜 인증 상태 관리에는 Context + `onAuthStateChange` 조합 추천.

4. Supabase Authentication vs profiles 테이블

Authentication 메뉴는 손댈 필요 없음. `supabase.auth.signUp()`을 호출하면 Supabase가 내부적으로 관리하는 `auth.users` 테이블에 이메일/비밀번호(해시)가 자동 저장됨. 직접 테이블을 그리는 게 아니라 Supabase가 알아서 처리.

문제는 회원가입 폼의 **추가 정보**(역할, Full Name, Phone)는 `auth.users`에 저장 안 됨 → 별도로 `profiles` 테이블을 만들어서 `auth.users.id`를 외래키로 연결해야 함.

만드는 방법:

- Supabase 대시보드 **Table Editor**에서 클릭으로 생성
- **SQL Editor**에 SQL문 실행 (아래 5번 참고)
- Supabase AI(대시보드 내 AI 어시스턴트)로 생성 요청도 가능

5. profiles 테이블 SQL 설계

파일: `src/services/supabase/schema.sql`

```
-- profiles 테이블: auth.users에 없는 추가 회원 정보 저장
create table public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  full_name text not null,
  phone text,
  role text not null check (role in ('tenant', 'landlord', 'realtor')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- RLS 활성화: 기본적으로 아무도 접근 못 하게 막고, 아래 정책으로만 허용
alter table public.profiles enable row level security;

create policy "본인 프로필 조회"
on public.profiles for select
```

```

using (auth.uid() = id);

create policy "본인 프로필 수정"
on public.profiles for update
using (auth.uid() = id);

create policy "본인 프로필 생성"
on public.profiles for insert
with check (auth.uid() = id);

-- 회원가입(auth.users insert) 시 profiles 행을 자동 생성하는 트리거
create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
insert into public.profiles (id, full_name, phone, role)
values (
new.id,
new.raw_user_meta_data ->> 'full_name',
new.raw_user_meta_data ->> 'phone',
coalesce(new.raw_user_meta_data ->> 'role', 'tenant')
);
return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();

```

구조 설명

- `id`는 `auth.users.id`를 그대로 참조 (1:1 관계, 회원 탈퇴하면 `on delete cascade`로 `profiles`도 같이 삭제)
- `role`은 signUp 페이지의 `UserRole` 타입(`tenant` / `landlord` / `realtor`)과 동일하게 `check` 제약
- RLS(Row Level Security)로 본인 데이터만 보고/고칠 수 있게 제한
- 트리거가 핵심: `signUp()` 호출 시 `full_name`, `phone`, `role`을 넘겨주면 자동으로 `profiles`에 복사됨

적용 방법: Supabase 대시보드 → SQL Editor에 위 내용을 붙여넣고 실행 (AI가 원격 DB에 직접 실행할 수는 없어서 대시보드에서 직접 돌려야 함).

6. signUp 폼 연결 순서 (8단계 로드맵)

단계	내용	이유
1	폼 입력값을 자바스크립트가 읽을 수 있게 만들기 (<code>FormData</code>)	<code><input></code> 은 그냥 화면 상자일 뿐, 제출 시 <code>FormData</code> 로 한 번에 걸어야 값을 읽을 수 있음

단계	내용	이유
2	<code>onSubmit + e.preventDefault()</code> 로 새로고침 막기	기본 동작은 폼 제출 시 페이지 새로고침. React에서 직접 처리하려면 막아야 함
3	<code>createClient()</code> 로 Supabase 도구 가져오기	<code>client.ts</code> 가 브라우저에서 Supabase 서버와 통신할 연결 객체를 만들어줌
4	<code>supabase.auth.signUp({...})</code> 호출	실제 회원가입 요청. 시간이 걸리므로 <code>async/await</code> 필요
5	<code>{ data, error }</code> 로 성공/실패 구분	성공하면 <code>data</code> , 실패하면 <code>error</code> 가 채워짐
6	로딩 상태(<code>isLoading</code>) 표시	중복 클릭/중복 가입 시도 방지
7	성공 시 다른 페이지로 이동 (<code>useRouter</code>)	가입 완료 후 로그인/홈 화면으로 안내
8	이메일 인증 안내 문구 (선택)	Supabase 기본 설정은 이메일 인증 필요 → 가입 직후 바로 로그인 안 될 수 있음

비유 정리

- 1단계: 설문지에 적기만 했지 아무도 안 걸어간 상태 → `FormData`가 "한 번에 걷는" 역할
- 2단계: 제출 버튼이 원래 하는 "페이지 새로고침"이라는 습관을 멈추게 함
- 4~5단계: 편의점 키오스크에서 주문 버튼 누르기 → "주문 성공"(`data`) 아니면 "품절"(`error`) 둘 중 하나로 응답

7. handleSubmit 함수 뜯어보기

`src/app/login/signUp/page.tsx`

```
const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  const formData = new FormData(e.currentTarget);
  const values = Object.fromEntries(formData);
  console.log("파인너츠", values);
};
```

줄	하는 일	쉬운 설명
<code>e.preventDefault();</code>	브라우저 기본 동작 막음	원래는 폼 제출 시 페이지가 새로고침되며 서버로 이동하려 함 → 그걸 막아서 React가 직접 처리하게 함
<code>const formData = new FormData(e.currentTarget);</code>	폼 안의 입력값들을 한 번에 모음	<code>e.currentTarget</code> 은 지금 제출된 그 <code><form></code> 자신. 이름/이메일/전화번호/비밀번호 값을 자동으로 다 긁어모음

줄	하는 일	쉬운 설명
<code>const values = Object.fromEntries(formData);</code>	모은 값을 보기 편한 형태로 변환	<code>FormData</code> 는 특수한 상자 모양이라 <code>{fullName: "홍길동", email: "a@a.com", ...}</code> 같은 평범한 객체로 바꿔줌
<code>console.log("파인너츠", values);</code>	콘솔에 값 출력 (테스트용)	아직 서버로 보내는 코드는 없고, "입력값이 잘 모였는지" 확인하는 임시 코드

8. 실제 회원가입 요청 보내기 (supabase.auth.signUp)

```
const supabase = createClient();
const { data, error } = await supabase.auth.signUp({
  email: values.email as string,
  password: values.password as string,
  options: {
    data: {
      full_name: values.fullName as string,
      phone: values.phone as string,
      role: selectedRole, // "tenant" | "landlord" | "realtor"
    },
  },
});

if (error) {
  console.error("회원가입 실패", error.message);
  return;
}

console.log("회원가입 성공", data);
```

줄	하는 일	쉬운 설명
<code>const supabase = createClient();</code>	Supabase와 연결할 "도구" 받기	<code>createClient()</code> 를 부르면 Supabase 서버랑 대화할 수 있는 "리모컨" 하나를 받음
<code>await supabase.auth.signUp({...})</code>	실제 회원가입 요청	Supabase가 미리 만들어둔 함수. 이메일/비밀번호를 넣어 부르면 서버에 "이 사람 가입시켜주세요" 요청. <code>await</code> 는 "응답 올 때까지 기다림"
<code>const { data, error } = ...</code>	응답을 두 조각으로 나눠 받기	성공하면 <code>data</code> (가입된 사용자 정보), 실패하면 <code>error</code> (실패 이유). 항상 둘 중 하나만 값 있음
<code>if (error) { ... return; }</code>	실패 처리	이메일 중복/비밀번호 짧음 등으로 <code>error</code> 발생 시 콘솔에 찍고 <code>return</code> 으로 함수 종료 → 아래 성공 코드 실행 안 됨

줄	하는 일	쉬운 설명
<code>console.log("회원가입 성공", data);</code>	성공 처리(임시)	지금은 콘솔에만 찍지만, 실제로는 "가입 완료! 다음 페이지로 이동" 처리가 들어감

비유: `signUp` 호출은 편의점 키오스크 주문 버튼과 비슷함. 버튼(함수)을 직접 만들지 않고 누르기(호출)만 함. 누른 다음엔 "주문 성공" 화면(`data`) 또는 "품질이예요"(`error`) 중 하나가 뜨고, 어느 쪽인지 확인해서 다르게 처리.

`options.data` — "추가 정보"를 같이 보내는 곳

`signUp`은 기본적으로 `email`, `password`만 받음. 이름/전화번호/역할처럼 추가로 입력받은 값은 `**options.data**`에 넣어서 같이 보내야 함.

왜 이렇게 해야 할까?

`schema.sql`의 트리거(`handle_new_user`)가 하는 일:

1. 새 회원이 가입하면(`auth.users`에 새 줄 생기면) 자동 실행
2. `new.raw_user_meta_data ->> 'full_name'` 처럼 `options.data`에 넣어 보낸 값을 key 이름으로 찾아서 꺼냄
3. 그 값을 `profiles` 테이블에 자동 저장

즉 `options.data`의 key 이름(`full_name`, `phone`, `role`)은 **DB 트리거가 찾는 이름과 정확히 똑같아야** 자동 저장됨. 이름이 다르면 빈 값 저장됨.

체크박스(약관 동의)는 왜 안 들어가나요?

`profiles` 테이블엔 약관 동의 저장 칸이 없음. `signUp`에 보내는 게 아니라, 보내기 **전에** "체크했는지"만 확인하는 용도로 씬.

```
if (!values.checkbox) {
  alert("약관에 동의해주세요.");
  return;
}
```

⚠ **체크해야 할 버그:** 지금 코드는 이 체크박스 검사가 `signUp` 호출 다음에 있어서, 체크 안 해도 이미 회원가입 요청이 서버로 나가버림. 검사 순서를 `signUp` 호출 **앞으로** 옮겨야 함.

비유: `options.data`는 택배 상자 겉면에 붙이는 "메모 스티커". 상자 안(이메일/비밀번호)은 정해진 자리가 있지만, 부가 정보는 스티커에 적어 같이 붙여 보내고, 받는 쪽(DB 트리거)이 스티커를 보고 알아서 분류.

9. 로딩 상태 관리 (isLoading)

```
const [isLoading, setIsLoading] = useState(false);

const handleSubmit = async (e: FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  const formData = new FormData(e.currentTarget);
```

```
const values = Object.fromEntries(formData);
const supabase = createClient();

setIsLoading(true); // 요청 시작 직전

try {
  const { data, error } = await supabase.auth.signUp({ /* ... */ });

  if (error) {
    console.error("회원가입 실패", error.message);
    return;
  }

  console.log("회원가입 성공", data);
} finally {
  setIsLoading(false); // 성공하든 실패하든 무조건 실행
}
};
```

- `await`로 기다리는 구간 앞뒤로 `true/false`를 켜고 꺼야 "로딩 중" 표시가 정확한 시간에만 보임
- `try/finally` 추천: `finally`는 "성공하든 실패하든 무조건 실행되는 청소 담당". `try` 없이 `return`을 여러 곳에서 쓰면 그때마다 `setIsLoading(false)`를 일일이 챙겨야 하고, 하나라도 빠뜨리면 로딩 스피너가 영원히 안 꺼지는 버그가 생김

10. 화면에 로딩 표시하기

버튼 텍스트 바꾸기 (추천)

```
<button
  type="submit"
  disabled={isLoading}
  className="... disabled:opacity-50"
>
  {isLoading ? "가입 처리 중..." : "Create Account"}
</button>
```

- `{isLoading ? "가입 처리 중..." : "Create Account"}` → 삼항연산자로 `true/false`에 따라 다른 텍스트
- `disabled={isLoading}` → 로딩 중엔 버튼 눌러도 반응 안 하게 잠금 (중복 클릭 방지, 이게 로딩 상태의 진짜 목적)

조건부로 문구 따로 표시하기

```
{isLoading && <p>가입 처리 중입니다...</p>}
```


- `isLoading`이 `true`면 `&&` 뒤의 `<p>`가 화면에 나타나고, `false`면 `&&`가 `false`를 반환해서 아무것도 안 그려짐 (React는 `false/null/undefined`를 화면에 그리지 않음)

11. `redirect()` vs `useRouter()`

	<code>redirect()</code> (next/navigation)	<code>useRouter()</code> (next/navigation)
어디서 쓰나	Server Component, Server Action, Route Handler (서버 실행 코드)	Client Component의 이벤트 핸들러 (버튼 클릭, 폼 제출 등)
동작 방식	서버가 브라우저에 "다른 페이지로 가라"고 응답을 보냄	브라우저(클라이언트)에서 직접 페이지 이동

지금 왜 `useRouter`를 써야 하나?

`signUp/page.tsx`는 "`use client`"가 붙어있고, `handleSubmit`도 브라우저에서 실행되는 이벤트 핸들러. `supabase.auth.signUp()`도 브라우저용 `client(client.ts)`를 사용. `redirect()`는 이벤트 핸들러(클릭 이벤트) 안에서 부르면 제대로 작동하지 않음 → `useRouter().push("/login")`이 맞는 방법.

비유: `redirect()`는 우체국 직원(서버)이 "이 편지 받으실 분은 이사 가셨어요" 하고 발신지에서 미리 안내하는 것. `useRouter()`는 내가 직접 차 몰고(브라우저) 이동하는 것. 지금은 이미 "내 차(클라이언트)"로 요청을 보낸 상황이라 도착도 내 차로 직접 가야 함.

현업에서는?

두 아키텍처 모두 혼함:

1. 클라이언트에서 `supabase-js` 직접 호출 + `useRouter` — 구조 단순, 빠르게 만들기 좋음 (지금 진행 중인 방식)
2. **Server Action**으로 처리 + `redirect()` — 폼의 `action`에 서버 함수 연결, 서버에서 `signUp` 처리 후 성공 시 서버에서 바로 `redirect()`. Supabase 공식 Next.js 가이드 권장 방식, 보안(비밀번호가 클라이언트 로직에 덜 노출)상 더 선호됨

추천: 지금은 client component 구조로 진행 중이니 `useRouter`로 계속. 로그인/회원가입 다 만든 후 "Server Action 리팩터링"을 별도 학습 코스로 해보는 것도 좋음.

12. 로그인 페이지 연결 순서

`src/app/login/page.tsx` — `signUp` 흐름을 거의 그대로 재사용하되, 딱 하나만 다름.

먼저 고쳐야 할 것: `name` 속성 누락

로그인 폼의 `<input>`은 `id`만 있고 `name`이 없음. `FormData`는 `name` 기준으로 값을 모으므로, `email/password` input에 `name="email"`, `name="password"`를 추가해야 함.

순서

단계	내용
1	<code><input></code> 에 <code>name</code> 속성 추가
2	<code>useState</code> 로 <code>isLoading</code> 상태 준비
3	<code><form></code> 에 <code>onSubmit</code> 연결 + <code>e.preventDefault()</code>
4	<code>FormData</code> 로 email/password 값 모으기
5	<code>createClient()</code> 로 supabase 도구 준비
6	<code>supabase.auth.signInWithPassword({ email, password })</code> 호출 (signUp과 다른 지점 — <code>options.data</code> 불필요)
7	<code>{ data, error }</code> 로 성공/실패 구분
8	에러 메시지를 화면에 표시 (아래 13번 참고)
9	성공하면 <code>useRouter().push("/")</code> 로 메인 페이지 이동
10	로딩 상태 표시 (<code>try/finally</code> , 버튼 <code>disabled</code> + 텍스트 변경 — signUp과 동일 패턴)

핵심: `onSubmit` 구조, `FormData` 처리, `isLoading` 관리, `useRouter` 이동 방식은 signUp 코드와 동일한 패턴. 바뀌는 건 `signUp()` → `signInWithPassword()`로 바꾸고 `options.data`를 빼는 것뿐.

13. 에러 메시지 상태 (errorMessage)

`isLoading`과 똑같은 방식 — "로그인 실패했을 때 보여줄 메시지"를 담는 상자.

```
const [errorMessage, setErrorMessage] = useState<string | null>(null);
```

- 처음엔 에러 없음 → `null`
- 로그인 실패 시 → `setErrorMessage("이메일 또는 비밀번호가 틀렸습니다")`

signUp 때 했던 것

```
if (error) { console.error(...);
return; }
```

콘솔에만 찍힘 (개발자만 봄)

로그인에서 추가할 것

```
if (error) { setErrorMessage("...");
return; }
```

상태에 저장 → 화면에 그려짐 (사용자가 봄)

화면 표시는 10번 항목에서 배운 조건부 렌더링(&&) 그대로 사용:

```
{errorMessage && <p className="text-red-500 text-sm">{errorMessage}</p>}
```

비유: `isLoading`이 "지금 로딩 중이야?"를 기억하는 상자였다면, `errorMessage`는 "실패 이유가 뭐였어?"를 기억하는 상자. 패턴(`useState` → 이벤트에서 값 변경 → JSX에서 조건부 렌더링)은 완전히 동일.

14. AuthContext로 로그인 상태를 앱 전체에 공유하기

왜 필요한가?

`useState`는 하나의 컴포넌트 안에서만 유효함. 로그인은 `/login` 페이지에서 일어나는데, 로그인 여부 표시는 `src/components/layout/header.tsx`처럼 모든 페이지에 공통으로 쓰는 컴포넌트에서 필요함. 서로 다른 컴포넌트가 같은 상태를 봐야 하므로 React의 **Context** 기능이 필요.

구조

`src/context/auth-context.tsx`

```
"use client";
import { createContext, useContext, useEffect, useState } from "react";
import { createClient } from "@/services/supabase/client";
import type { User } from "@supabase/supabase-js";

const AuthContext = createContext<User | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const supabase = createClient();

  useEffect(() => {
    supabase.auth.getUser().then(({ data }) => setUser(data.user));

    const { data: listener } = supabase.auth.onAuthStateChange(
      (_event, session) => setUser(session?.user ?? null),
    );

    return () => listener.subscription.unsubscribe();
  }, []);

  return <AuthContext.Provider value={user}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  return useContext(AuthContext);
}
```

동작 순서

1. `useState<User | null>(null)`로 현재 유저를 저장
2. 컴포넌트가 처음 뜰 때(`useEffect`) `supabase.auth.getUser()`로 "지금 로그인 돼 있나?" 최초 확인
3. `supabase.auth.onAuthStateChange(...)`로 로그인/로그아웃이 언제 어디서 일어나든 자동 감지해서 상태 갱신 — 핵심! `/login`에서 로그인 성공하는 순간 이 리스너가 반응해서 값이 바뀜
4. `src/app/layout.tsx`에서 `<AuthProvider>`로 `<Header />`와 `{children}`을 감싸서 앱 전체에 적용
5. `header.tsx`에서 `const user = useAuth();` 한 줄로 로그인 여부를 어디서든 꺼내 씀

이 패턴이 이 프로젝트에서만 쓰는 건가, 필드 공통인가?

- `createContext/useContext/Provider` → **React 자체 기능**, 어떤 프로젝트든 100% 동일하게 재사용 가능
- `AuthProvider + useAuth()` 구조 → 로그인 상태 관리에 **업계에서 거의 표준처럼 쓰는 패턴**, 그대로 다른 프로젝트에 복사해도 됨
- 프로젝트마다 바뀌는 건 딱 `supabase.auth.getUser()`, `onAuthStateChange()`, `createClient()` 세 줄 뿐. Firebase/Auth0/Clerk 등 다른 인증 서비스를 쓰면 이 부분만 그 서비스 API로 교체하면 되고, 나머지 뼈대 (Context 생성 → Provider로 감싸기 → 훅으로 꺼내 쓰기)는 그대로 유지됨
- 참고: 실무에서는 이걸 직접 안 짜고 `NextAuth.js(Auth.js)`, `Clerk` 같은 라이브러리로 대체하는 경우도 많음. 직접 만들어보면 그 라이브러리들이 내부적으로 뭘 하는지 이해하게 됨

15. 트러블슈팅: useAuth is not a function

증상

```
TypeError: (0 , _context_auth_context__WEBPACK_IMPORTED_MODULE_3___useAuth) is not a function
```

원인 확인 결과: `auth-context.tsx`에 `useAuth`가 정상적으로 export 돼 있고, import 경로(`@/context/auth-context`)도 정확했음. 코드 자체는 문제 없음.

진짜 원인: 새 파일(`auth-context.tsx`)을 만든 시점에 Next.js 개발 서버가 이미 켜져 있으면, Webpack Fast Refresh(HMR)가 새 모듈을 완전히 못 잡아내고 옛날(비어있던) 버전을 캐싱해버리는 경우가 있음.

해결 순서

1. 개발 서버 껐다가 다시 켜기 (`Ctrl+C` → `npm run dev`)
2. 그래도 안 되면 `.next` 캐시 폴더 삭제 후 재시작 — 캐시일 뿐이라 지워도 안전하고 다음 빌드 때 자동 재생성됨

참고: 이 에러를 해결해도, `layout.tsx`에서 아직 `<AuthProvider>`로 앱을 감싸지 않았다면 `useAuth()`가 항상 `null`을 반환함. 그건 별개로 `layout.tsx`에서 `<Header />`와 `{children}`을 `<AuthProvider>`로 감싸야 해결됨.

16. Header에 로그인/로그아웃 버튼 조건부 표시하기

`src/components/layout/header.tsx`에 이미 `const user = useAuth();`로 로그인 유저 정보를 갖고 있는 상태에서, 로그인/로그아웃 버튼을 상황에 맞게 바꾸는 작업.

순서

1. `createClient()` import 추가 — 로그아웃도 결국 Supabase에 요청을 보내야 하므로 (`@/services/supabase/client`)
2. `handleLogout` 함수 만들기 — 그 안에서 `supabase.auth.signOut()` 호출
3. 조건부 렌더링으로 버튼 분기

```
{user ? (
  <button onClick={handleLogout}>로그아웃</button>
) : (
```

```
<Link href="/login" className="login-button">로그인</Link>
  )}
```

- `user`가 있으면(로그인 상태) → 로그아웃 버튼
 - `user`가 `null`이면(로그아웃 상태) → 기존 로그인 링크
4. 로그아웃은 `<Link>`가 아니라 `<button>` — 페이지 이동이 아니라 "동작(요청)"을 실행하는 것이므로 `onClick` 핸들러가 있는 `<button>`이 맞음
 5. `setUser`를 직접 안 건드려도 되는 이유 — `auth-context.tsx`에 이미 만들어둔 `onAuthStateChange` 리스너가 있기 때문. `signOut()`을 부르면 Supabase가 "로그아웃 됐다"는 이벤트를 발생시키고, 그 리스너가 자동 감지해서 `setUser(null)`을 실행해줌. 그래서 `handleLogout` 함수 안에서는 `signOut()` 호출 한 줄이면 끝
 6. (선택) 로그아웃 후 페이지 이동 — 보호된 페이지에 있었다면 메인으로 보내는 게 자연스러움. `useRouter()`의 `router.push("/")`를 `signOut()` 다음 줄에 추가

정리: Context + 리스너 패턴을 미리 만들어둔 덕분에, 로그아웃 버튼 쪽 코드는 "Supabase에 로그아웃 요청 보내기"만 하면 되고, 화면 갱신은 Context가 알아서 처리함.